COURSE PROJECT REPORT

Fogline: An Adaptive Multi-Vector Cyber Threat Prediction Framework Using Temporal Graph Intelligence

Submitted by

Aswanth V (23BCE2030)

Sharan Mahendran (23BCE2310)

Course: [Course Code] – [Course Name]
Slot: [Slot]

Under the guidance of

Prof. Govinda K

School of Computer Science and Engineering (SCOPE)

VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

October 2026

Title

Fogline: An Adaptive Multi-Vector Cyber Threat Prediction Framework Using Temporal Graph Intelligence

Aim

To build a cyber threat prediction framework that detects attacks with light models at the edge, sends only difficult cases to a resource-aware fog layer, and uses a temporal graph of multi-vector security events to predict the next attack stage and the next likely victim.

Objectives

1. To detect attacks on the real ToN-IoT network dataset using Logistic Regression, Decision Tree and Random Forest models at the edge.
2. To design an edge to fog rule that sends only uncertain cases to a stronger model, and picks the fog model based on CPU and RAM load.
3. To build a dynamic security graph where users, hosts, servers and external IPs are nodes and network, identity, endpoint and alert events are timestamped edges.
4. To build a temporal graph model with adaptive vector weights that predicts the next attack stage and the next victim host.
5. To compare the model with Random Forest, XGBoost, LSTM and static graph baselines using macro F1, accuracy, Hits@k, MRR and early-warning lead time.
6. To split all data by time so that models never learn from the future.

Abstract

Modern cyberattacks happen in several steps. An attacker first scans the network, then steals passwords, runs malicious programs, moves to other machines and finally steals data or damages services. Most intrusion detection systems only check whether a single event is an attack, so they miss how the attack is growing over time and across machines. They also often run heavy models on every event, which is costly for small devices. In this project we built Fogline, a framework that detects attacks cheaply and also predicts what the attacker is likely to do next. Fogline has three layers. The edge layer runs three light models, Logistic Regression, Decision Tree and Random Forest, on each connection. Only when these models are unsure or disagree is the connection sent to the fog layer, which picks a suitable model based on the current CPU and RAM load. We followed the full data science process of data cleaning, exploratory analysis, feature building, model training and time based evaluation. The third layer is a temporal graph intelligence layer. Here, users, hosts, servers and external IP addresses are nodes, and network, login, endpoint and alert events are timestamped edges between them. Our temporal graph model, called TG-AMV, looks at the recent events around the current

victim, gives each kind of evidence an adaptive weight, and predicts the next attack stage and the next host that is likely to be attacked. We tested detection on the real ToN-IoT network dataset after removing IP addresses, text fields and more than 117,000 duplicate rows. The Random Forest reached 98.89% accuracy, and it named the exact attack type with 96.9% accuracy. When we applied the edge to fog rule to ToN-IoT, only 2.1% of connections needed the fog layer and accuracy stayed the same as running the Random Forest on every connection. Public datasets do not provide multi-vector attack episodes with stage labels in a ready form, so for the prediction tasks we built a synthetic temporal security graph with 32,674 events, 480 attack incidents and four evidence vectors, and split it by time. For next stage prediction, TG-AMV reached 75.4% accuracy against 73.0% for Random Forest, 71.8% for XGBoost and 75.1% for an LSTM, while always guessing the most common stage gave only 23.7%. It warned about data theft or damage before it happened in 39.6% of test incidents, with a median warning time of about 26 minutes, while the Random Forest gave no early warnings. For predicting the next victim during lateral movement, the temporal graph model reached a mean reciprocal rank of 0.227, compared with 0.170 for a static graph and 0.096 for random guessing. The learned vector weights also made sense, for example network evidence received the highest weight before data exfiltration. These results show that time and graph structure add real value for predicting attacks, especially for finding the next victim, though the gain for stage prediction over a good sequence model was small. The main limits are that the prediction results come from synthetic data and that the full timestamped ToN-IoT files could not be used in this work.

Keywords: *cyber threat prediction, temporal graph, multi-stage attacks, ToN-IoT, edge and fog computing, next victim prediction*

1. Introduction

Cyberattacks today rarely happen in one step. Frameworks such as the cyber kill chain and MITRE ATT&CK describe how an attacker scans a network, gets access, moves between machines and finally steals data or causes damage. Each step leaves small traces in different places, such as a burst of port scans, a few failed logins or an unusual program running on a server. On their own these traces may look harmless, but together they show a clear path.

Traditional intrusion detection systems look at one event or one short window of traffic and ask whether it is an attack. This is useful, but it misses the links between events on different machines and the way an attack grows over time.

A security team gains much more if the system can also say what is likely to happen next. If we can predict that an attacker will move to a database server, the team can protect that server before the attack reaches it.

Security data naturally forms a graph. Users log into hosts, hosts talk to servers, processes run on machines and alerts point to targets. Each of these links has a time. A temporal graph keeps both the structure and the timing of these events. For example, if a user who recently logged into an infected laptop also logs into a file server, that server becomes a likely next target. A normal table of features cannot easily show this kind of link.

Graph neural networks have become popular for intrusion detection, and recent work has moved from static graphs to dynamic and temporal graphs. However, most of this work still focuses on detection inside one kind of data, usually network traffic.

Evidence about an attack comes from several sources, which we call vectors. These are network flows, login events, endpoint activity and security alerts. The most useful vector changes as the attack moves from one stage to the next, so a model should be able to change how much it trusts each one.

Detection must also be practical. Running a heavy model on every connection is costly for small edge devices. Fogline uses light models at the edge and sends only uncertain cases to a stronger model in the fog layer, chosen according to the current system load. Earlier studies have shown that simple models such as decision trees give a good balance of accuracy, speed and power use on small devices, so they are a sensible first line of defence.

Testing must be done carefully. If parts of the same attack appear in both training and test data, or if the model sees future data during training, the results look better than they really are. We split all data by time and by incident, and we removed duplicate rows and identity columns such as IP addresses from the real dataset.

The main contributions of this project are an edge to fog detection pipeline tested on the real ToN-IoT dataset, a multi-vector temporal security graph, the TG-AMV model with adaptive

vector weights for next stage and next victim prediction, and a fair comparison with Random Forest, XGBoost, LSTM and static graph baselines.

Section 2 reviews related work, Section 3 explains the proposed methodology, Section 4 presents and discusses the results, and Section 5 gives the conclusion and future work.

2. Literature Review

Zhong et al. [1] surveyed graph neural network based intrusion detection and grouped the work into graph construction, model design and deployment. They showed that graphs capture communication links that table based models ignore, but most surveyed systems only detect attacks and do not predict them.

Friji et al. [2] studied multi-stage attack detection and prediction on ToN-IoT using a three stage version of the kill chain. Each stage had its own model, and outputs of earlier stages were used to predict later ones, with an average F1 score of about 94%. Their stages are fixed, while our model predicts the next stage from an evolving temporal graph.

Song et al. [3] built DGIDS, which models in-vehicle CAN traffic as a dynamic graph and updates graph features in real time. Van Langendonck et al. [4] proposed PPT-GNN, a spatio-temporal graph network pre-trained without labels that works on short traffic windows. Both focus on detection rather than predicting the attacker's next step.

Freitas and Gharib [5] described TITAN, a large dynamic threat-intelligence graph used at Microsoft that spreads reputation scores to find risky entities. It shows that dynamic graphs work at industry scale, but it scores entities rather than following a single incident step by step.

Wang et al. proposed BS-GAT [6], a graph attention model for intrusion detection in edge computing, and FedSTGCN [7], which combines spatio-temporal graph learning with federated learning so IoT devices can train together without sharing raw data. Both support using graphs at the edge, which matches the edge and fog design of Fogline.

Zhang et al. [8] combined a graph convolution network with Transformers for in-vehicle networks, and Zhao et al. [10] built DMSTG-AD, a dynamic multi-scale spatio-temporal graph model for software defined networks. Yao and Koirala [11] added fault tolerance to temporal graph detection for IIoT and 5G. Tanvir et al. [12] used autoencoder pretraining with a spatio-temporal graph network for healthcare IoT. All of these report very high detection scores but remain focused on detection in one domain.

The closest work to ours is by Makki Nayeri and Rezvani [9], who treated alert prediction as temporal link prediction between attackers and victims using a Temporal Graph Network with a Transformer based aggregator, which lets them identify the next victim. Their later BiTA model [15] extends this idea. Our next victim task follows the same idea but uses multi-vector evidence inside an organisation.

For lateral movement, Khoury et al. [13] proposed Jbeil, which applies temporal graph learning to login graphs and finds lateral movement as a link prediction problem. King and Huang [14] built Euler, which combines a graph network with a recurrent network for the same task. These works show that time-aware graphs help predict where an attacker moves next.

Our model borrows two ideas from foundational temporal graph work, namely node memory from TGN [19] and time encoding with attention from TGAT [20]. We also follow the advice of Arp et al. [17] on avoiding data leakage, and the findings of Tekin et al. [18] that simple models such as decision trees are efficient on small devices.

For data, ToN-IoT [16] was recorded on a testbed with IoT, edge, fog and cloud layers, which matches our design. Table 1 summarises the reviewed work. Across these papers, no single system combines cheap edge detection, load-aware fog selection, adaptive multi-vector evidence and prediction of both the next stage and the next victim, which is the gap Fogline addresses.

Table 1. Summary of the reviewed work

Paper	What they did	Gap compared with this project
[1] Zhong et al., 2024	Survey of GNN based IDS	Mostly detection, not prediction
[2] Friji et al., 2024	Staged GNN prediction on ToN-IoT	Fixed three stage design
[3] Song et al., 2024	Dynamic graph IDS for CAN	Vehicle domain, detection only
[4] Van Langendonck et al., 2024	Pre-trained spatio-temporal GNN	Detection focus
[5] Freitas & Gharib, 2025	Dynamic threat-intelligence graph	Entity risk, not incident steps
[6], [7] Wang et al., 2025	Graph attention and federated ST-GNN	Detection at the edge
[8], [10]–[12] 2025–2026	Spatio-temporal GNN detectors	Single domain detection
[9] Makki Nayeri & Rezvani, 2026	TGN alert and next victim prediction	Alerts only, no stages
[13] Khoury et al., 2024	Temporal graph lateral movement	Login graph only
[14] King & Huang, 2023	GNN + RNN link prediction	Lateral movement only
This project (Fogline)	Edge-fog detection + TG-AMV	Combines detection, stage and victim prediction

3. Proposed Methodology

3.1 System Architecture

Fig. 1 shows the three layers of Fogline.

- **Edge layer.** Each connection is checked by three light models, Logistic Regression, Decision Tree and Random Forest. A risk score is also calculated. If the models agree and are confident, the decision is made here.
- **Fog layer.** Uncertain connections come here. The system reads the current CPU and RAM load and picks the best model that the machine can afford at that moment.
- **Temporal graph intelligence layer.** Every event is stored as a timestamped edge in a dynamic security graph. The TG-AMV model reads the graph around the current victim and predicts the next attack stage and the next likely victim, together with a confidence score and an early warning.

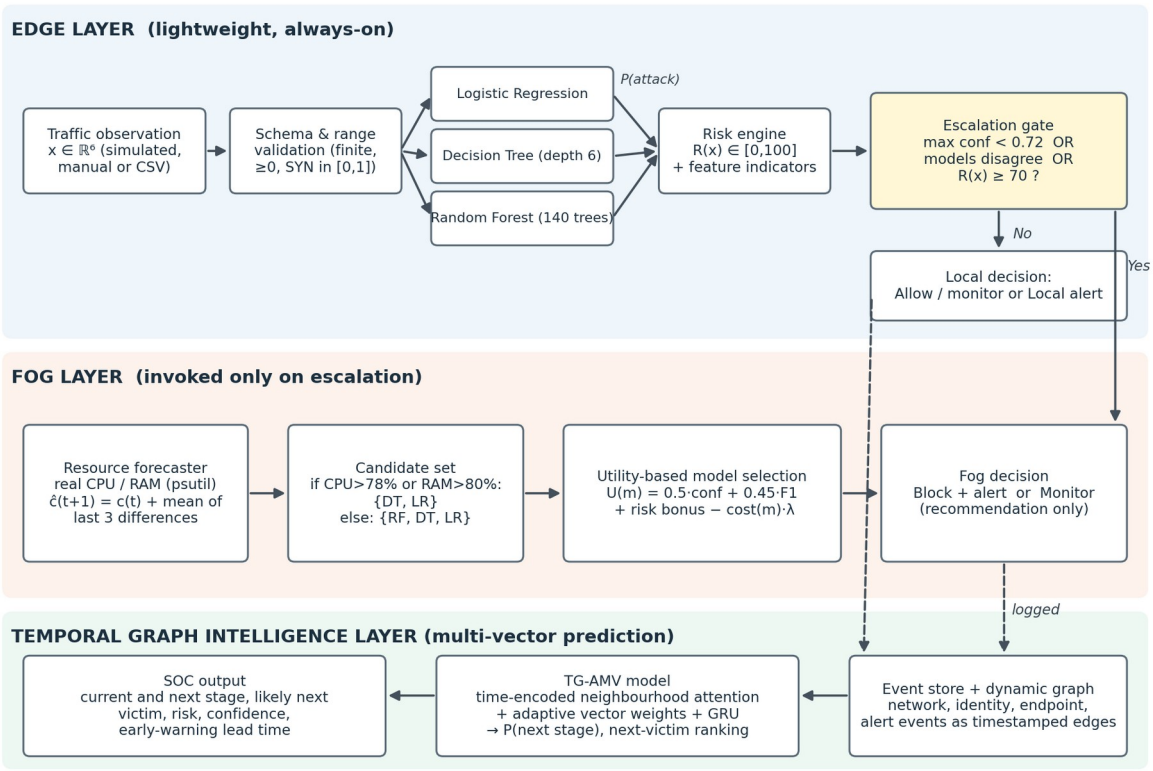


Fig. 1. Architecture of Fogline with edge, fog and temporal graph intelligence layers

3.2 Project Workflow

We followed the data science workflow in Fig. 2. We collected and generated data, cleaned and explored it, split it by time, built features and graphs, trained the models, evaluated them and connected them to the Fogline application.

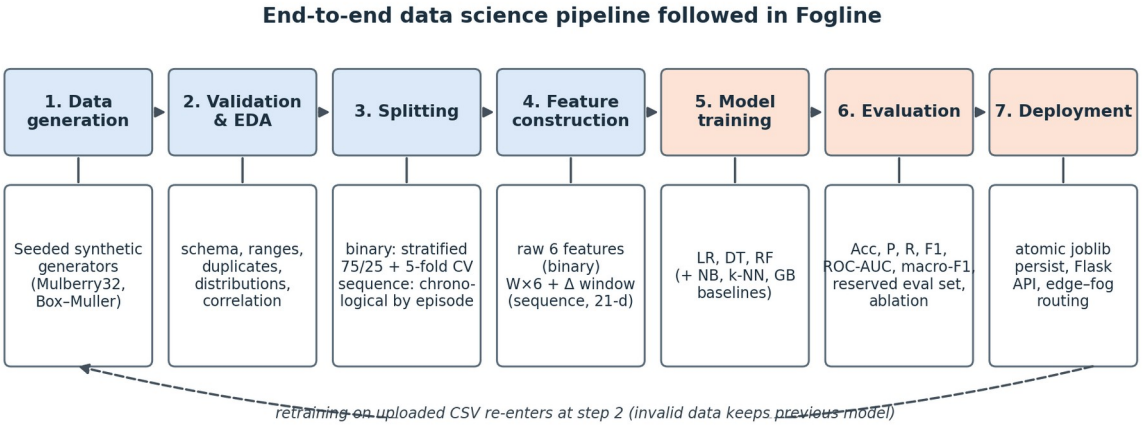


Fig. 2. Block diagram of the project workflow

3.3 Datasets

We used three datasets, listed in Table 2.

Table 2. Datasets used in the project

Dataset	Type	Size	Used for
ToN-IoT network (train-test version) [16]	Real	211,043 rows, 44 columns	Edge detection, attack type, edge to fog test
Multi-vector temporal security graph	Synthetic	32,674 events, 480 incidents	Next stage and next victim prediction
Fogline traffic samples	Synthetic	6,000 rows	Resource-aware fog model selection demo

ToN-IoT network dataset

ToN-IoT was recorded at UNSW Canberra on a testbed with IoT devices, edge and fog servers and cloud services. Each row is one network connection, labelled as normal or as one of nine attack types, namely backdoor, DDoS, DoS, injection, password attack, ransomware, scanning, XSS and man-in-the-middle (MITM). In the copy we used, normal traffic had been reduced to 50,000 rows. We cleaned it in these steps.

- Removed IP address and port columns, because in this testbed the attacker machines have fixed addresses and the model could recognise the machine instead of the attack.
- Removed text columns such as DNS query names, URLs and certificate names.
- Converted true or false columns to 1 and 0, and one-hot encoded protocol, service and connection state, giving 47 features.
- Removed 117,373 duplicate rows, leaving 93,670 unique rows, so the same row cannot appear in both training and testing.
- Split into 75% training (70,252 rows) and 25% testing (23,418 rows), keeping the share of each attack type the same.

Multi-vector temporal security graph

The train-test version of ToN-IoT has no timestamps, and public datasets do not provide attack incidents with stage labels across login, endpoint and network data in a ready form. So, following the prototype plan from our first review, we generated a temporal security graph that follows MITRE ATT&CK style stages. Table 3 describes it. Attacks follow six realistic paths, for example reconnaissance, credential access, execution, lateral movement and exfiltration. During lateral movement the attacker moves to hosts that share a user with the current victim or talk to it often, so the next victim depends on the graph. Who logs in where also changes every ten days, so recent history matters more than old history.

Table 3. Contents of the temporal security graph

Item	Details
Nodes	253 in total: 40 hosts (4 servers), 30 users and external attacker IPs
Edges (events)	32,674 timestamped events over 60 days, of which 6,674 belong to 480 attack incidents
Evidence vectors	network 15,736, identity 9,469, endpoint 6,204, alert 1,265
Attack stages	reconnaissance, credential access, execution, lateral movement, exfiltration, impact
Event features	packet rate, failed logins, unique ports, packet size, duration, SYN ratio, bytes out, privileged process

Fig. 3 shows the graph around one test incident. The bold edges are attack events and the thin edges are normal activity on the same machines.

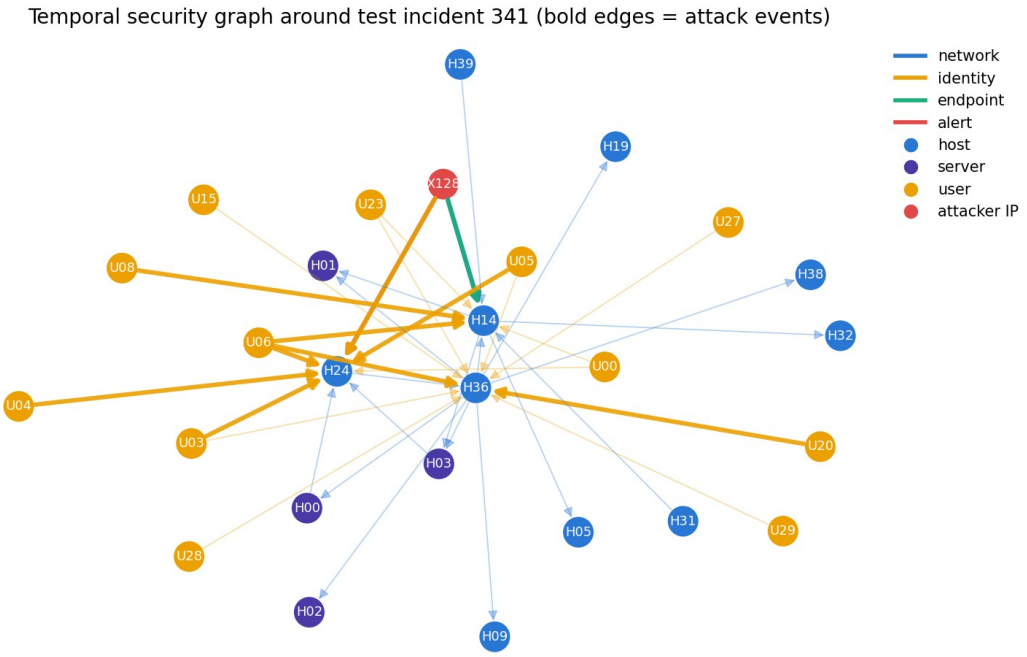


Fig. 3. Part of the temporal security graph around one test incident

3.4 Edge Detection Models

Logistic Regression gives the attack probability using the sigmoid function. **Decision Tree** splits the data using the Gini impurity, where p_k is the share of class k. **Random Forest** averages the answers of T trees.

$$p(\text{attack}) = 1 \,/\, (1 + e\hat{}-(w \cdot x + b)) \tag{1}$$

$$\text{Gini} = 1 - \Sigma\, p_k{}^2 \tag{2}$$

$$p_RF(x) = (1/T)\ \Sigma\, p_t(x) \tag{3}$$

3.5 Edge to Fog Decision

A connection is sent to the fog layer if the highest model confidence is below 0.72, if the three models give different answers, or if the risk score is 70 or more. In the fog layer, if the forecast CPU is above 78% or RAM is above 80%, the heavy Random Forest is not allowed. Each allowed model m gets a score and the highest score wins.

$$\text{Score}(m) = 0.5 \times \text{confidence} + 0.45 \times \text{F1} - \text{cost penalty} \tag{4}$$

3.6 TG-AMV Temporal Graph Model

TG-AMV stands for Temporal Graph model with Adaptive Multi-Vector weights. It takes ideas from TGN [19] and TGAT [20] but is kept small enough for a student project. For each prediction point at time t, it works in five steps.

1. **Temporal neighbourhood.** Take the last 24 events that touch the current victim host or the attacker, from any vector, up to time t. Each event e has features x_e and an age $\Delta t_e = t - t_e$.
2. **Time encoding.** The age of each event is turned into a set of waves, so the model can tell recent events from old ones.

$$\varphi(\Delta t) = [\cos(\omega_1 \Delta t), \ldots, \cos(\omega_8 \Delta t), \sin(\omega_1 \Delta t), \ldots, \sin(\omega_8 \Delta t)] \tag{5}$$

$$h_e = \text{MLP}([x_e \,;\, \varphi(\Delta t_e)]) \tag{6}$$

3. **Attention per vector.** For each vector v (network, identity, endpoint, alert), the events of that vector are combined using attention weights α_e.

$$\alpha_e = \text{softmax}(a^T h_e), \qquad s_v = \Sigma\, \alpha_e\, h_e \tag{7}$$

4. **Adaptive vector weights.** A small layer looks at the four vector summaries and at g, a GRU summary of the incident's last six events, and decides how much to trust each vector.

$$\beta = \text{softmax}(W\ [s_net \,;\, s_id \,;\, s_end \,;\, s_alert \,;\, g]), \qquad z = \Sigma\, \beta_v\, s_v \tag{8}$$

5. **Prediction.** The fused vector z and g are passed to a final layer that gives the probability of each next stage.

$$P(\text{next stage}) = \text{softmax}(\text{MLP}([z \,;\, g])) \tag{9}$$

For **next victim prediction**, every host that is not yet compromised is a candidate. For each candidate c we compute time-decayed links with the current victim, where recent events count more. These include shared recent logins, recent network traffic, recent alerts and whether c is a server. A logistic model trained on past incidents ranks the candidates.

$$w(\text{victim}, c) = \Sigma_e \exp(-(t - t_e) / \tau), \qquad \tau = 3 \text{ days} \tag{10}$$

3.7 Baselines and Fair Testing

We compared TG-AMV with the baselines planned in our first review: always guessing the most common stage, a Random Forest and an XGBoost model on the last three events (the original Fogline method), an LSTM on the last six events, and a static graph model that adds fixed host statistics from past data to XGBoost. For next victim prediction we compared with random guessing, picking the most attacked host, and a static graph that uses old login and traffic counts without time.

All prediction data was split by time. The earliest 70% of incidents were used for training and the rest for testing, and any training incident that ended after the first test incident started was removed. This gave 335 training and 144 test incidents, with 3,760 and 1,457 prediction points. Fig. 4 shows the idea of windows and time based splitting.

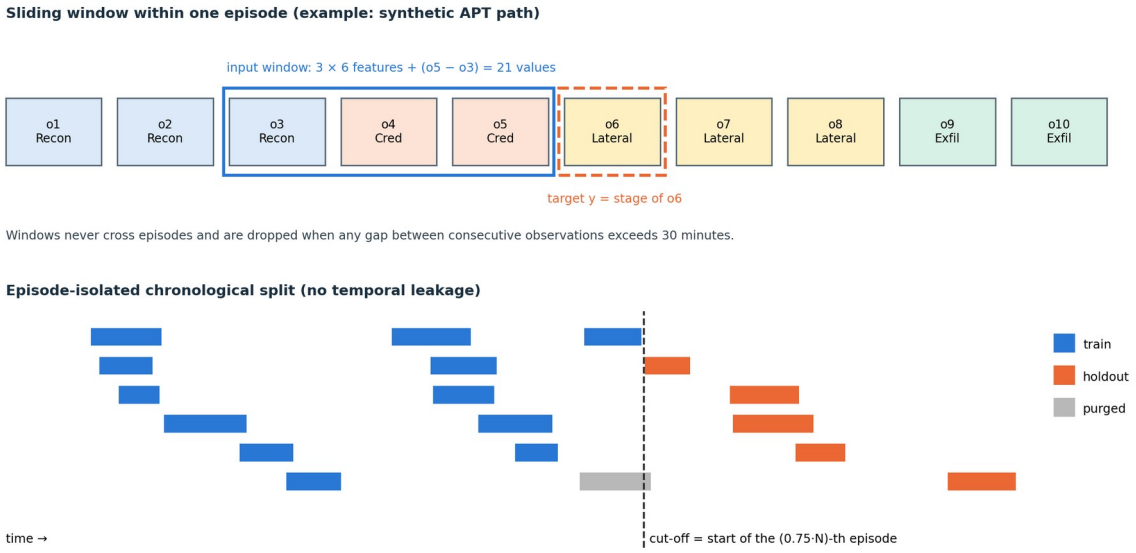


Fig. 4. How recent events form an input window (top) and how incidents are split by time (bottom)

3.8 Evaluation Metrics

For detection and stage prediction we used accuracy, precision, recall, F1 and macro F1, which is the average F1 over all classes. For next victim prediction we used Hits@k, the share of cases where the true victim is in the top k, and mean reciprocal rank (MRR). We also measured **early-warning lead time**, the time between the first warning of exfiltration or impact and the moment it actually started.

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall}) \tag{11}$$

$$\mathrm{MRR} = (1/\mathrm{N})\ \Sigma_{\mathrm{i}}\ 1\ /\ \mathrm{rank}_{\mathrm{i}} \tag{12}$$

4. Results and Discussion

4.1 Exploring ToN-IoT

Fig. 5 shows the traffic types after cleaning. More than half of the rows were duplicates, which shows how important this step is. The data is unbalanced, with many injection and password attacks but only 1,039 MITM and 260 ransomware rows.

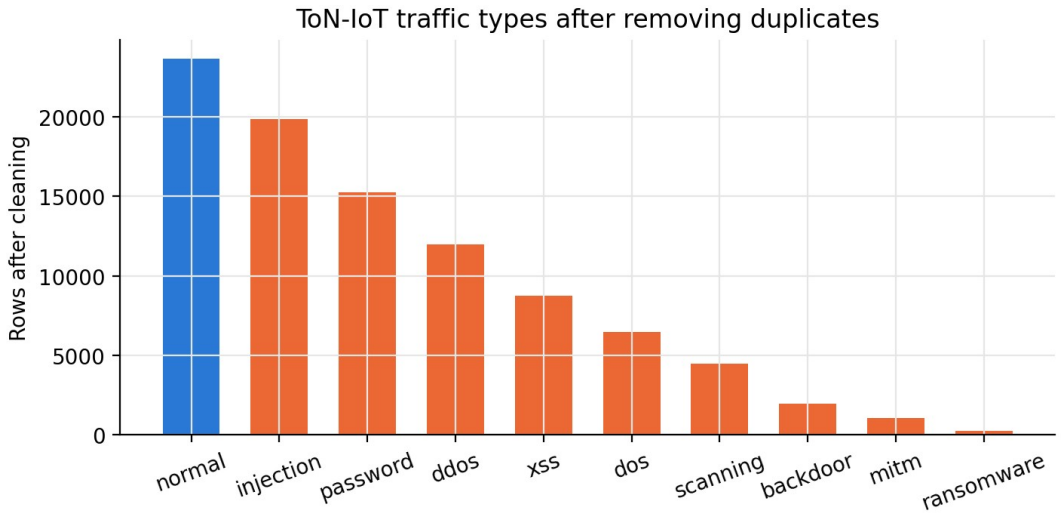


Fig. 5. ToN-IoT traffic types after removing duplicates

4.2 Attack Detection on ToN-IoT

Table 4 shows the edge models on the ToN-IoT test set of 23,418 connections.

Table 4. Attack detection on the ToN-IoT test set (all 47 features)

Model	Accuracy	Precision	Recall	F1	False alarm rate	5-fold CV accuracy
Logistic Regression	97.34%	0.983	0.982	0.982	5.1%	97.39% ± 0.06
Decision Tree	98.70%	0.987	0.996	0.991	3.9%	98.75% ± 0.12
Random Forest	98.89%	0.988	0.997	0.993	3.5%	99.01% ± 0.06

The Random Forest was best with 98.89% accuracy and caught at least 97.3% of every attack type. Cross-validation results were almost the same, so the result is stable. When we used only the six simple features of the original Fogline app, the Random Forest still reached 96.06% accuracy but its false alarm rate rose to 14.7%, so richer features are needed for real traffic (Fig. 6).

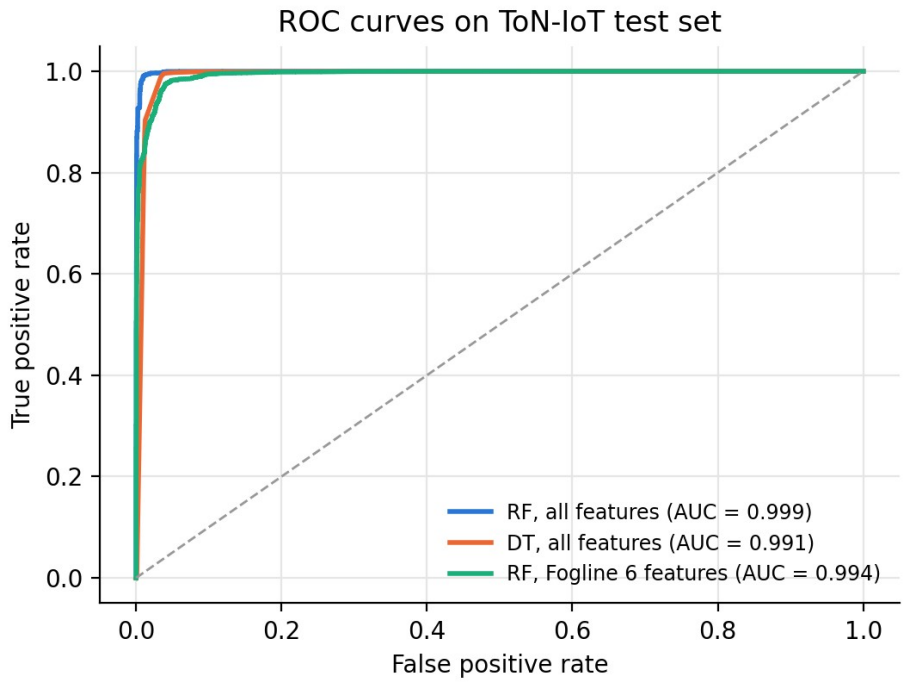


Fig. 6. ROC curves on ToN-IoT with all features and with the six Fogline features

A Random Forest trained to name the exact attack type reached 96.9% accuracy and 0.936 macro F1 (Fig. 7). MITM (F1 0.726) and ransomware (F1 0.897) were weakest because they have the fewest rows, and some DoS and DDoS connections were mixed up.

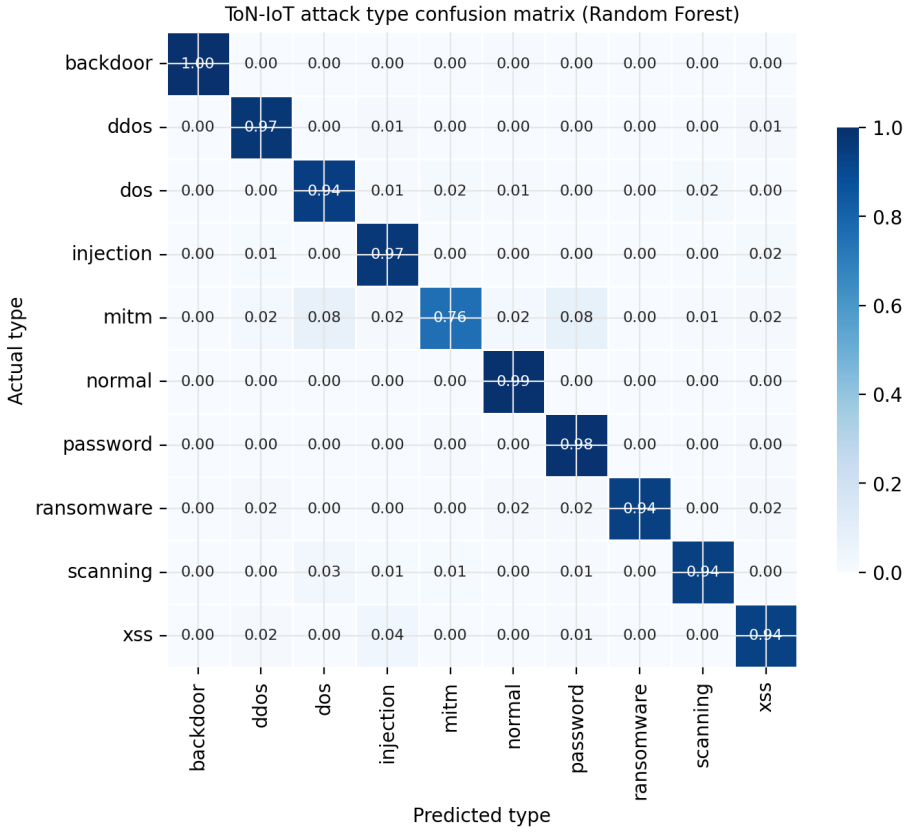


Fig. 7. Confusion matrix for the ten ToN-IoT traffic types

4.3 Edge to Fog Decision

Table 5 shows the edge to fog rule on the ToN-IoT test set. Only 2.1% of connections went to the fog layer. On those hard cases the best edge model was right 76.5% of the time, while the fog Random Forest was right 89.5% of the time. Overall accuracy was 98.89%, the same as running the Random Forest on everything, but the heavy model was needed for only about 2 in every 100 connections.

Table 5. Edge to fog rule on the ToN-IoT test set

Measure	Value
Connections kept at the edge	22,924 (97.9%)
Connections sent to the fog layer	494 (2.1%)
Accuracy of edge decisions	99.1%
Accuracy on sent connections, best edge model	76.5%
Accuracy on sent connections, fog Random Forest	89.5%
Overall accuracy with the rule	98.89%

To test the resource-aware part, we passed 1,500 Fogline samples through the system under normal and busy conditions (Fig. 8). When the machine was not busy, the fog layer mostly picked the Random Forest (499 times). When it was busy, it never picked the Random Forest and used Logistic Regression 531 times instead.

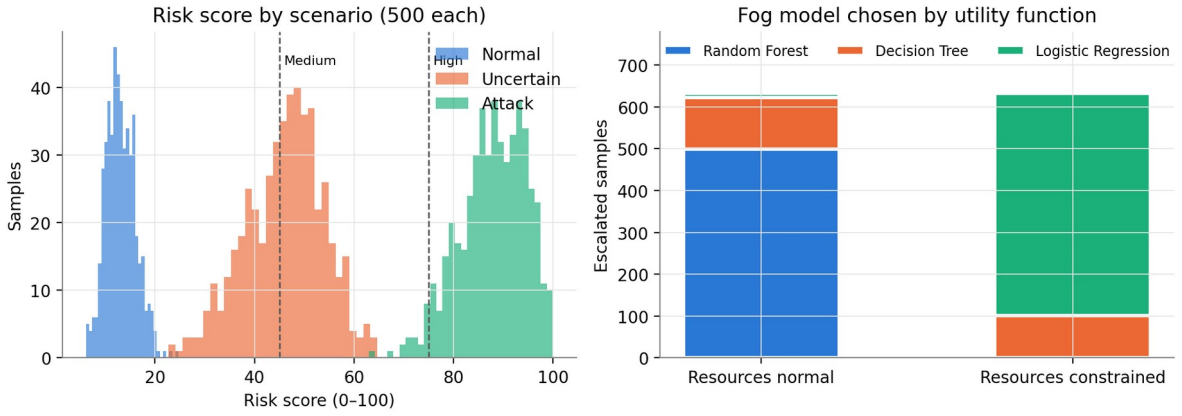


Fig. 8. Risk scores by sample type (left) and fog model chosen under normal and busy conditions (right)

4.4 Next Attack Stage Prediction

Table 6 and Fig. 9 compare all models on the 1,457 test prediction points.

Table 6. Next attack stage prediction on test incidents

Model	Accuracy	Macro F1
Majority class	23.7%	0.064
Random Forest (window)	73.0%	0.730
XGBoost (window)	71.8%	0.710
Static graph + XGBoost	71.4%	0.710
LSTM (sequence)	75.1%	0.753

Model	Accuracy	Macro F1
Temporal graph (TG-AMV, ours)	75.4%	0.744

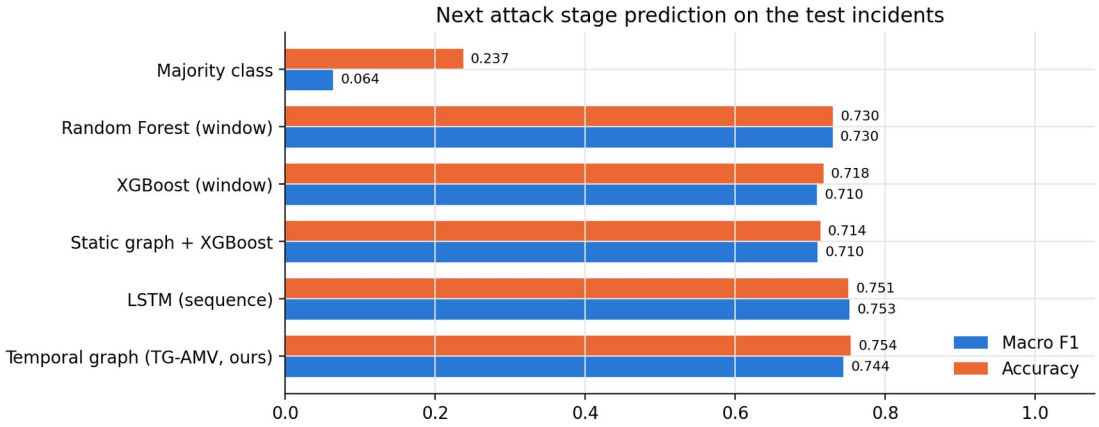


Fig. 9. Accuracy and macro F1 of all next stage models

TG-AMV had the highest accuracy (75.4%), and the LSTM had a slightly higher macro F1 (0.753 against 0.744). Both models that use the order of events beat the window models (Random Forest and XGBoost) by 2 to 4 points. Adding fixed host statistics (static graph) did not help. When we removed the graph neighbourhood from TG-AMV at test time, accuracy fell to 73.0%, so the time-aware graph context added about 2.5 points. Fig. 10 shows that most errors happen between neighbouring stages, for example execution and lateral movement.

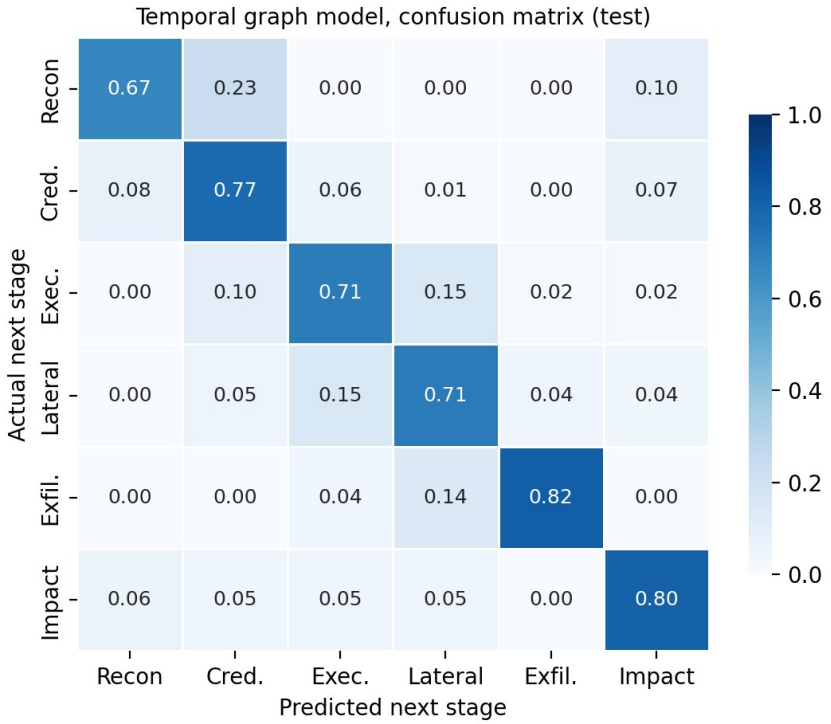


Fig. 10. Confusion matrix of TG-AMV on the test incidents

Table 7. F1 score for each next stage

Next stage	Random Forest (window)	TG-AMV (ours)
Reconnaissance	0.633	0.635
Credential access	0.687	0.727
Execution	0.680	0.717
Lateral movement	0.691	0.714
Exfiltration	0.838	0.863
Impact	0.852	0.810

4.5 Adaptive Vector Weights

Fig. 11 shows the average weight TG-AMV gave to each evidence vector, grouped by the current stage. The weights change with the stage, as planned in our first review. Network evidence got the highest weight during exfiltration (0.77) and impact (0.51), when large transfers and floods happen. Identity evidence got the highest weight during credential access (0.51) and execution (0.56), the steps just before the attacker logs into a new machine. This makes the model easier to explain to an analyst.

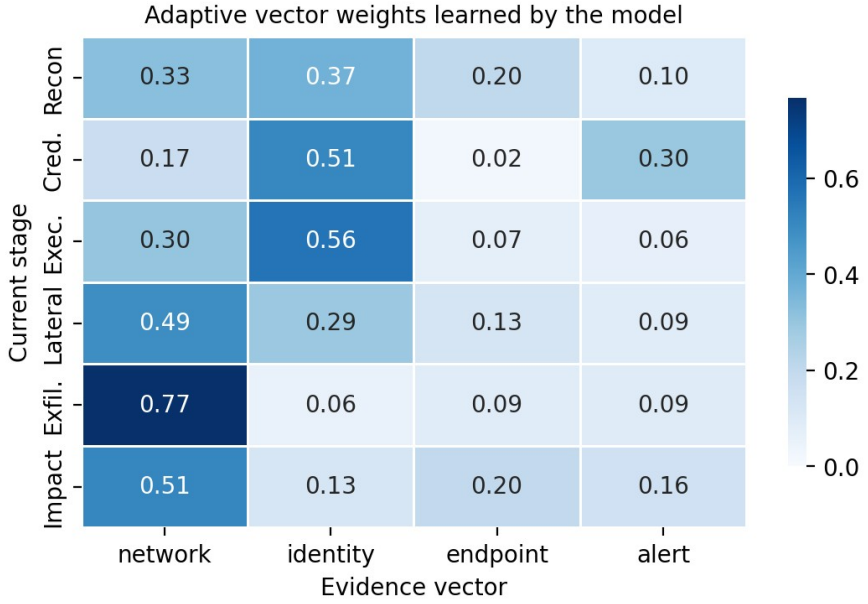


Fig. 11. Average adaptive vector weights for each current stage

4.6 Next Victim Prediction

Table 8 shows how well each method ranked the next victim host during lateral movement (102 test moves) and over all moves (148).

Table 8. Next victim prediction results

Method	Lateral: Hits@1	Lateral: Hits@3	Lateral: MRR	All moves: MRR
Random guess	0.020	0.069	0.096	0.107
Most attacked host (popularity)	0.069	0.147	0.162	0.348

Method	Lateral: Hits@1	Lateral: Hits@3	Lateral: MRR	All moves: MRR
Static graph	0.059	0.157	0.170	0.354
Temporal graph (ours)	0.088	0.196	0.227	0.357

During lateral movement, the temporal graph reached an MRR of 0.227, against 0.170 for the static graph and 0.096 for random guessing, an improvement of about 34% over the static graph. This is because the hosts a user logged into recently are better clues than old history. Over all moves, the three graph and popularity methods were close, because many moves during exfiltration go to the database or file server, which is easy to guess (Fig. 12).

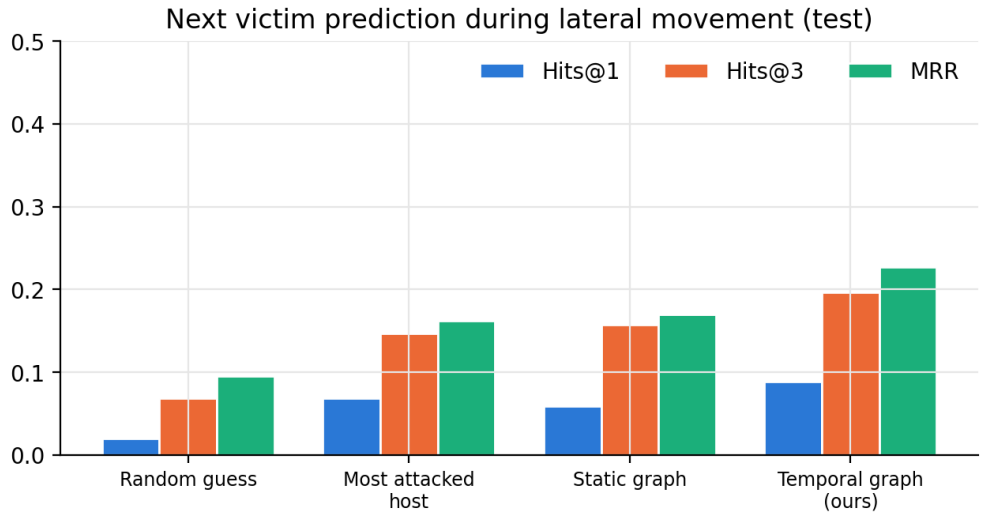


Fig. 12. Next victim prediction during lateral movement

4.7 Early Warning

An early warning is given when the model's probability of exfiltration or impact as the next stage goes above 0.5 before it actually starts. Table 9 shows the result over the 144 test incidents that reached one of these stages.

Table 9. Early warning before exfiltration or impact

Model	Incidents warned	Warning rate	Median lead time (minutes)
Random Forest (window)	0 of 144	0.0%	0.0
LSTM (sequence)	57 of 144	39.6%	20.3
Temporal graph (TG-AMV, ours)	57 of 144	39.6%	25.6

The Random Forest never crossed the threshold, because its probabilities were spread over many classes. TG-AMV and the LSTM both warned in 39.6% of incidents, and TG-AMV warned earlier, with a median of 25.6 minutes against 20.3 minutes.

4.8 Comparison with Related Work

Table 10 lists results reported in related papers next to ours. The datasets and tasks are different, so the numbers cannot be compared directly, but they show where our work stands.

Table 10. Reported results in related work and in this project

Work	Data	Task	Reported result
Friji et al. [2]	ToN-IoT	Staged attack prediction	Average F1 about 94%
BS-GAT [6]	Public IDS data	Binary detection	Above 99% accuracy
Tanvir et al. [12]	Healthcare IoT to ToN-IoT	Cross-dataset detection	95.1% accuracy
Yao & Koirala [11]	ToN-IoT and others	Detection under faults	Macro F1 at least 92.4%
Fogline (ours)	ToN-IoT, cleaned	Binary detection	98.89% accuracy
Fogline (ours)	ToN-IoT, cleaned	Attack type	96.9% accuracy
Fogline TG-AMV (ours)	Temporal security graph	Next stage	75.4% accuracy

4.9 Discussion

- The edge to fog design worked well on real data. Only 2.1% of connections needed the heavy model, with no loss of accuracy.
- Time and graph structure helped most for next victim prediction during lateral movement, where the temporal graph beat the static graph by about a third.
- For next stage prediction, the gain of TG-AMV over a good LSTM was small. Most of the signal comes from the incident's own recent events, and the graph neighbourhood added about 2.5 points.
- The adaptive vector weights changed sensibly with the attack stage, which supports the multi-vector idea from our first review and makes the output easier to explain.
- The prediction results come from synthetic data, and TG-AMV is a simplified version of TGN and TGAT. The full timestamped ToN-IoT and CIC-IDS2017 files could not be downloaded in our environment, so real-data testing of the prediction layer is left for future work.

5. Conclusion

In this project we built Fogline, an adaptive multi-vector cyber threat prediction framework that joins cheap edge detection with temporal graph intelligence. The edge layer runs Logistic Regression, Decision Tree and Random Forest models and sends a connection to the fog layer only when they are unsure or disagree. The fog layer reads the CPU and RAM load and picks a lighter model when the system is busy. The temporal graph layer stores network, login, endpoint and alert events as timestamped edges between users, hosts, servers and external IPs, and our TG-AMV model uses this graph to predict the next attack stage and the next victim. On the real ToN-IoT dataset, after removing identity columns and duplicate rows, the Random Forest reached 98.89% accuracy and named the attack type with 96.9% accuracy. The edge to

fog rule sent only 2.1% of connections to the fog layer and still matched the accuracy of running the Random Forest everywhere, which supports the main idea of saving resources without losing accuracy. On the temporal security graph, TG-AMV reached 75.4% accuracy for next stage prediction, better than Random Forest, XGBoost and a static graph model and about level with an LSTM. It gave early warning before data theft or damage in 39.6% of test incidents with a median warning time of about 26 minutes, while the Random Forest gave none. For next victim prediction during lateral movement, the temporal graph features gave a mean reciprocal rank of 0.227, about one third better than the static graph, because who logged in where recently tells more than old history. The adaptive vector weights were also easy to read, for example network evidence counted most before exfiltration and identity evidence counted most around credential access. We also learned some honest lessons. The gain of the temporal graph model over a good sequence model for stage prediction was small, and most of its benefit came from the incident's own recent events. The prediction experiments use synthetic data, because public datasets do not provide multi-vector incidents with stage labels in a ready form, and the full timestamped ToN-IoT files could not be downloaded in our environment. The resource forecast is a simple trend rule, and our temporal graph model is a simplified version of TGN and TGAT rather than a full implementation. In the future we plan to build the temporal graph from the full ToN-IoT and CIC-IDS2017 files with timestamps and IP addresses, compare with a full TGN, add the LANL login data for lateral movement, calibrate the model probabilities, and connect the prediction layer to the Fogline dashboard so an analyst sees the predicted next stage and victim in real time. We also saw that careful data cleaning matters, because removing duplicate rows and identity columns from ToN-IoT gave a more honest picture of model performance. The Decision Tree remains a good choice for busy edge devices because it is fast and small. Working on this project showed us that predicting attacks needs good data structure, careful time based testing and honest baselines just as much as a strong model.

References

[1] M. Zhong, M. Lin, C. Zhang, and Z. Xu, "A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges," Computers & Security, vol. 141, Art. no. 103821, Jun. 2024, doi: 10.1016/j.cose.2024.103821.

[2] H. Friji, I. Mavromatis, A. Sanchez-Mompo, P. Carnelli, A. Olivereau, and A. Khan, "Multi-stage attack detection and prediction using graph neural networks: An IoT feasibility study," arXiv preprint arXiv:2404.18328, Apr. 2024.

[3] J. Song, G. Qin, Y. Liang, J. Yan, and M. Sun, "DGIDS: Dynamic graph-based intrusion detection system for CAN," Computers & Security, vol. 147, Art. no. 104076, 2024, doi: 10.1016/j.cose.2024.104076.

[4] L. Van Langendonck, I. Castell-Uroz, and P. Barlet-Ros, "PPT-GNN: A practical pre-trained spatio-temporal graph neural network for network security," arXiv preprint arXiv:2406.13365, Jun. 2024.

[5] S. Freitas and A. Gharib, "Web scale graph mining for cyber threat intelligence," in Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD '25), Toronto, Canada, Aug. 2025, pp. 4447–4456, doi: 10.1145/3711896.3737274.

[6] Y. Wang, Z. Han, Y. Du, J. Li, and X. He, "BS-GAT: A network intrusion detection system based on graph neural network for edge computing," Cybersecurity, vol. 8, Art. no. 27, 2025, doi: 10.1186/s42400-024-00296-8.

[7] Y. Wang, J. Li, Z. Han, P. Cheng, and R. Kumar, "FedSTGCN: A novel federated spatiotemporal graph learning-based network intrusion detection method for the Internet of Things," Frontiers of Information Technology & Electronic Engineering, vol. 26, no. 7, pp. 1164–1179, 2025, doi: 10.1631/FITEE.2400932.

[8] J. Zhang, X. Fan, and Z. Zhao, "A hybrid intrusion detection model based on dynamic spatial-temporal graph neural network in in-vehicle networks," Scientific Reports, vol. 15, Art. no. 34736, Oct. 2025, doi: 10.1038/s41598-025-18401-3.

[9] Z. Makki Nayeri and M. Rezvani, "Alert prediction in computer networks using transformer-based temporal graph neural networks: Identifying the next victim," Journal of Network and Computer Applications, vol. 249, Art. no. 104455, May 2026, doi: 10.1016/j.jnca.2026.104455.

[10] J. Zhao, D. Zhang, Q. He, M. Lin, and Y. Yang, "DMSTG-AD: An SDN intrusion detection method based on dynamic multi-scale spatio-temporal graph neural network," Scientific Reports, vol. 16, Art. no. 14528, Mar. 2026, doi: 10.1038/s41598-026-44360-4.

[11] J. Yao and B. Koirala, "Defense-aware temporal graph neural network for fault-tolerant intrusion detection in IIoT–5G environments," Cluster Computing, vol. 29, no. 5, Art. no. 319, Jun. 2026, doi: 10.1007/s10586-026-06037-5.

[12] M. I. M. Tanvir, N. Y. Nadia, H. R. Rabby, M. H. Arif, K. Nur, and D. Ghose, "Spatial–temporal graph neural network with autoencoder pretraining for intrusion detection in healthcare IoT ecosystems," Scientific Reports, vol. 16, Art. no. 15000, Mar. 2026, doi: 10.1038/s41598-026-45041-y.

[13] J. Khoury, Đ. Klisura, H. Zanddizari, G. De La Torre Parra, P. Najafirad, and E. Bou-Harb, "Jbeil: Temporal graph-based inductive learning to infer lateral movement in evolving enterprise networks," in Proc. IEEE Symposium on Security and Privacy (SP), 2024, pp. 3644–3660, doi: 10.1109/SP54263.2024.00009.

[14] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," ACM Transactions on Privacy and Security, vol. 26, no. 3, Art. no. 35, 2023, doi: 10.1145/3588771.

[15] Z. Makki Nayeri and M. Rezvani, "BiTA: Bidirectional gated recurrent unit-transformer aggregator in a temporal graph network framework for alert prediction in computer networks," arXiv preprint arXiv:2604.22781, Apr. 2026.

[16] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," Sustainable Cities and Society, vol. 72, Art. no. 102994, 2021, doi: 10.1016/j.scs.2021.102994.

[17] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Pitfalls in machine learning for computer security," Communications of the ACM, vol. 67, no. 11, pp. 104–112, 2024, doi: 10.1145/3643456.

[18] N. Tekin, A. Acar, A. Aris, A. S. Uluagac, and V. C. Gungor, "Energy consumption of on-device machine learning models for IoT intrusion detection," Internet of Things, vol. 21, Art. no. 100670, 2023, doi: 10.1016/j.iot.2022.100670.

[19] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," arXiv preprint arXiv:2006.10637, 2020.

[20] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in Proc. Int. Conf. Learning Representations (ICLR), 2020.

Appendix: Tools and Code

Table 11. Tools and libraries

Tool	Used for
Python, pandas, NumPy	Data loading, cleaning and graph building
scikit-learn, XGBoost	Edge models and baselines
PyTorch	LSTM and TG-AMV models
matplotlib, seaborn, NetworkX	Charts and graph drawing
Flask, SQLite, HTML/CSS/JS	Fogline web application and event store
psutil	Reading CPU and RAM load

Results can be reproduced with toniot_experiments.py (ToN-IoT detection and edge to fog test), temporal_graph_experiments.py (graph generation, TG-AMV and baselines) and experiments.py (resource-aware fog selection). All random seeds are fixed.